> CSS styling works at any API level. With Toolkit DSL, elements automatically support CSS classes and IDs. For lower-level APIs, implement `Styleable` and pass a `styleResolver` to widgets—see Implementing Styleable and the `css-no-toolkit-demo` for examples.

TamboUI supports CSS-based styling as an alternative to programmatic style configuration. Instead of chaining `.bold().cyan().onBlue()` calls, you can define styles in external `.tcss` files and apply them via selectors.

Why Use CSS?

Programmatic styling works well for simple cases:

```
Element programmaticExample = column(
    text("Hello").bold().cyan(),
    panel("Title", () -> text("content")).rounded().borderColor(Color.BLUE)
);
```

But as applications grow, CSS offers advantages:

- **Separation of concerns** - styling lives outside your code
- **Theming** - switch between light/dark themes at runtime
- **Designer-friendly** - non-developers can adjust colors and spacing
- **Consistency** - define styles once, apply everywhere via classes

The StyleEngine

`StyleEngine` manages stylesheets and resolves styles for elements:

```
StyleEngine engine = StyleEngine.create();

// Add inline CSS
engine.addStylesheet("Panel { border-type: rounded; }");

// Load named themes from classpath
engine.loadStylesheet("dark", "/themes/dark.tcss");
engine.loadStylesheet("light", "/themes/light.tcss");

// Switch themes at runtime
engine.setActiveStylesheet("dark");
```

TCSS Format

TamboUI uses `.tcss` (TamboUI CSS) files - a CSS dialect designed for terminal UIs.

Variables

Define reusable values:

```
$bg-primary: black;
$fg-primary: white;
$accent: cyan;
$border-color: dark-gray;

Panel {
    background: $bg-primary;
    color: $fg-primary;
    border-color: $border-color;
}

Panel:focus {
    border-color: $accent;
}
```

Selectors

Type selectors match element types:

```
Panel {
    border-type: rounded;
}

Text {
    color: white;
}
```

Class selectors match CSS classes:

```
.primary {
    color: cyan;
    text-style: bold;
}

.danger {
    color: red;
}
```

ID selectors match specific elements:

```
#sidebar {
    width: 30;
    background: dark-gray;
}

#main-content {
    padding: 1;
}
```

Pseudo-class selectors match element state:

```
Panel:focus {
    border-color: cyan;
    border-type: double;
}
```

```
Button:hover {
    text-style: bold;
}

ListElement-item:selected {
    background: blue;
}

ListElement-item:nth-child(even) {
    background: #1a1a1a;
}
```

Supported pseudo-classes: `:focus`, `:hover`, `:disabled`, `:active`, `:selected`, `:first-child`, `:last-child`, `:nth-child(even)`, `:nth-child(odd)`.

Compound selectors combine multiple conditions (no space between parts):

```
Panel.primary#main {
    border-color: cyan;
}
```

> Whitespace matters in selectors:

```
/* Text element WITH class "muted" (compound - no space) */
Text.muted {
    color: gray;
}

/* Element with class "muted" INSIDE a Text element (descendant - with space) */
Text .muted {
    color: gray;
}

/* Text element INSIDE an element with class "muted" */
.muted Text {
    color: gray;
}
```

Descendant and child combinators:

```
/* Any Button inside a Panel */
Panel Button {
    color: white;
}

/* Direct child only */
Panel > Button {
    text-style: bold;
}
```

Selector lists (comma-separated) apply the same styles to multiple selectors:

```
/* Apply same styles to multiple selectors */
.error, .warning, .danger {
    text-style: bold;
}

Panel.primary, Panel.secondary {
    border-type: rounded;
}
```

Attribute selectors match elements based on their attributes (title, label, placeholder):

```
/* Match elements with a specific attribute value */
Panel[title="Settings"] {
    border-color: cyan;
}

/* Match elements that have an attribute (any value) */
Panel[title] {
    border-type: double;
}

/* Starts with */
Panel[title^="Test"] {
    border-color: yellow;
}
```

```
/* Ends with */
Panel[title$="Output"] {
    border-color: green;
}

/* Contains */
Panel[title*="Tree"] {
    border-color: magenta;
}
```

Attribute selectors can be combined with other selectors:

```
/* Attribute selector with class */
Panel.sidebar[title="Navigation"] {
    border-color: cyan;
    border-type: double;
}

/* Attribute selector with pseudo-class */
TextInputElement[placeholder]:focus {
    border-color: yellow;
}

/* Nested elements inside a panel with specific title */
Panel[title="Settings"] TextInputElement {
    border-type: rounded;
}

/* Direct child with attribute */
Panel[title="Form"] > TextInputElement[placeholder="Enter name..."] {
    border-color: green;
}

/* Multiple attribute selectors */
TextInputElement[title="Username"][placeholder] {
    color: cyan;
}

/* Selector list with attribute selectors */
Panel[title="Input"], Panel[title="Output"], Panel[title="Logs"] {
    border-color: blue;
}
```

```
/* Style child elements based on parent's attribute */
Panel[title^="Test"] GaugeElement {
    color: yellow;
}

/* Combine with ID selector */
#main-panel[title="Dashboard"] {
    border-type: double;
    border-color: cyan;
}
```

Elements expose the following attributes for CSS matching:

Element	Available Attributes
`Panel`	`title` , `bottom-title`
`DialogElement`	`title`
`ListElement`	`title`
`TableElement`	`title`
`TabsElement`	`title`
`GaugeElement`	`title` , `label`
`LineGaugeElement`	`label`
`SparklineElement`	`title`
`ChartElement`	`title`

Element	Available Attributes
`BarChartElement`	`title`
`CalendarElement`	`title`
`CanvasElement`	`title`
`TextInputElement`	`title` , `placeholder`
`TextAreaElement`	`title` , `placeholder`

Targeting Elements: Practical Examples

This section shows common patterns for targeting specific elements in your UI.

Styling elements by location

```
/* All text elements inside the sidebar */
#sidebar Text {
    color: gray;
}

/* Only direct children of the sidebar */
#sidebar > Text {
    text-style: bold;
}

/* Gauges inside any panel with class "metrics" */
Panel.metrics GaugeElement {
    color: green;
}

/* Text inside a list inside a panel */
Panel ListElement Text {
```

```
    color: cyan;
}
```

Styling elements by type and class

```
/* All panels with "card" class */
Panel.card {
    border-type: rounded;
    padding: 1;
}

/* Primary buttons vs danger buttons */
Button.primary {
    color: cyan;
    text-style: bold;
}

Button.danger {
    color: red;
    text-style: bold;
}

/* Muted text anywhere */
Text.muted {
    color: gray;
    text-style: dim;
}
```

Styling based on state

```
/* Focused input fields */
TextInputElement:focus {
    border-color: cyan;
}

/* Selected items in any list */
ListElement-item:selected {
    background: blue;
    text-style: bold;
}
```

```
/* Alternating row colors in tables */
TableElement-row:nth-child(even) {
    background: #1a1a1a;
}

/* First and last items */
ListElement-item:first-child {
    border-color: yellow;
}

ListElement-item:last-child {
    border-color: yellow;
}
```

Combining conditions

```
/* Focused panel with primary class */
Panel.primary:focus {
    border-color: cyan;
    border-type: double;
}

/* Selected item inside a focused list */
ListElement:focus ListElement-item:selected {
    background: cyan;
    color: black;
}

/* Specific element by ID when focused */
#username-input:focus {
    border-color: green;
}

/* Text with multiple classes */
Text.title.large {
    text-style: bold;
    color: white;
}
```

Styling child components

Many elements have styleable sub-components accessed via child selectors:

```
/* Style the filled portion of gauges */
GaugeElement-filled {
    color: green;
}

/* Style the unfilled portion of line gauges */
LineGaugeElement-unfilled {
    color: dark-gray;
}

/* Style list items */
ListElement-item {
    color: white;
}

/* Style selected tab */
TabsElement-tab:selected {
    text-style: bold reversed;
}

/* Style table header */
TableElement-header {
    text-style: bold;
    color: cyan;
}

/* Style cursor in text input */
TextInputElement-cursor {
    text-style: reversed;
    background: yellow;
}

/* Style placeholder text */
TextInputElement-placeholder {
    color: gray;
    text-style: italic;
}
```

Scoped styling with selector lists

```
/* Apply same border style to multiple panel types */
#header, #footer, #sidebar {
    border-type: rounded;
```

```
    border-color: gray;
}

/* Multiple element types with same styling */
GaugeElement, LineGaugeElement, SparklineElement {
    color: cyan;
}

/* Multiple states */
TextInputElement:focus, TextAreaElement:focus {
    border-color: yellow;
}

/* Complex selector list */
Panel.card Text.title,
Panel.card Text.subtitle,
DialogElement Text.title {
    text-style: bold;
}
```

Nesting

Use `&` for nested rules:

```
Panel {
    border-type: rounded;
    border-color: gray;

    &:focus {
        border-color: cyan;
        border-type: double;
    }

    &.primary {
        border-color: blue;
    }
}
```

Properties

Style Properties

Property	Values	Example
`color`	Named colors, hex, rgb	`color: cyan;` `color: #ff5500;`
`background`	Named colors, hex, rgb	`background: black;`
`text-style`	bold, dim, italic, underlined, reversed	`text-style: bold;`
`border-type`	plain, rounded, double, thick	`border-type: rounded;`
`border-color`	Named colors, hex, rgb	`border-color: cyan;`
`padding`	Single value or top right bottom left	`padding: 1;` `padding: 1 2 1 2;`
`text-align`	left, center, right	`text-align: center;`
`width`	fit, fill	`width: fit;`

Layout Properties

These properties control how elements are sized and positioned within their containers.

Property	Values	Example
`height`	Constraint for vertical sizing (used by Column)	`height: 5;` `height: fill;` `height: 50%;`
`width`	Constraint for horizontal sizing (used by Row)	`width: 10;` `width: fit;` `width: 25%;`

Property	Values	Example
`flex`	Flex positioning mode - see Flex Layout below for details	`flex: center;` `flex: space-between;`
`spacing`	Gap between children in cells	`spacing: 1;`
`margin`	Margin around element (single value or top right bottom left)	`margin: 2;` `margin: 1 2 1 2;`
`direction`	Layout direction for panels	`direction: horizontal;` `direction: vertical;`
`column-count`	Number of columns in ColumnsElement	`column-count: 3;`
`column-order`	Ordering mode for ColumnsElement children	`column-order: row-first;` `column-order: column-first;`
`grid-size`	Grid dimensions for GridElement — columns only or columns and rows	`grid-size: 3;` `grid-size: 3 4;`
`grid-columns`	Per-column width constraints for GridElement (space-separated, cycles when fewer than columns)	`grid-columns: fill fill(2) 20;`
`grid-rows`	Per-row height constraints for GridElement (space-separated, cycles when fewer than rows)	`grid-rows: 2 3;`
`grid-gutter`	Gutter spacing for GridElement — uniform or horizontal/vertical	`grid-gutter: 2;` `grid-gutter: 1 2;`

Property	Values	Example
`grid-template-areas`	CSS grid-template-areas style layout with named regions that can span multiple cells	`grid-template-areas: "H H H; S M M; F F F";`
`dock-top-height`	Height constraint for DockElement top region	`dock-top-height: 3;`
`dock-bottom-height`	Height constraint for DockElement bottom region	`dock-bottom-height: 3;`
`dock-left-width`	Width constraint for DockElement left region	`dock-left-width: 20;`
`dock-right-width`	Width constraint for DockElement right region	`dock-right-width: fill;`
`content-align`	Content alignment for StackElement children	`content-align: center;` `content-align: top-left;` `content-align: stretch;`
`flow-row-spacing`	Vertical spacing between rows in FlowElement	`flow-row-spacing: 1;`

Grid Template Areas

The `grid-template-areas` property enables CSS grid-template-areas style layouts where named regions can span multiple rows and columns. This is useful for complex dashboard layouts.

Template format:

- Use semicolon-separated rows: `"header header; nav main; footer footer"`
- Or quoted strings (CSS format): `"header header" "nav main" "footer footer"`

- Use `.` for empty cells
- Area names must start with a letter (alphanumeric and underscores allowed)
- Named areas must form contiguous rectangles

```
/* Dashboard layout with spanning regions */
.dashboard {
    grid-template-areas: "header header header; nav main main; nav main main; foot
    grid-gutter: 1;
}
```

```
// Combine CSS template with programmatic area assignment
Element gridExample = grid()
    .addClass("dashboard")
    .area("header", text("Dashboard").bold())
    .area("nav", list("Menu 1", "Menu 2"))
    .area("main", content)
    .area("footer", text("Status").dim());
```

> The template can be defined in CSS, but widgets must be assigned to areas programmatically using `.area(name, element)`. CSS alone cannot bind elements to named areas.

Constraint Values

The `height` and `width` properties accept constraint values:

Value	Description
`<number>`	Fixed size in cells (e.g., `height: 5;`)
`<number>%`	Percentage of available space (e.g., `width: 50%;`)
`fill`	Fill available space with weight 1 (e.g., `height: fill;`)

Value	Description
`fill(<weight>)`	Fill with specified weight (e.g., `height: fill(2);`)
`<n>fr`	Fractional unit — alias for `fill(<n>)` (e.g., `1fr` is `fill(1)` , `2fr` is `fill(2)`)
`fit`	Size to content (e.g., `width: fit;`)
`min(<value>)`	Minimum size (e.g., `height: min(3);`)
`max(<value>)`	Maximum size (e.g., `height: max(10);`)
`<n>/<d>`	Ratio (e.g., `width: 1/3;`)

Flex Layout

Flex layout controls how remaining space is distributed among children in `Row` and `Column` containers. When children don't fill the entire container (e.g., fixed-size elements in a large space), flex determines where they're positioned and how gaps are distributed.

Understanding Flex

Think of flex as answering: "What do I do with the extra space?"

- If your children have fixed sizes and don't fill the container, flex positions them
- Row containers use flex horizontally (left-to-right)
- Column containers use flex vertically (top-to-bottom)
- Flex only affects distribution of *remaining* space after sizing constraints are applied

Flex Modes

Value	Description	Use Case
`start`	Pack items at the start, remaining space at end	Left-aligned toolbars, top-aligned menus
`center`	Center items, equal space on both sides	Centered dialogs, hero content
`end`	Pack items at the end, remaining space at start	Right-aligned status indicators
`space-between`	First/last items at edges, equal gaps between	Navigation bars, spread-out controls
`space-around`	Equal space around each item (half-size at edges)	Evenly distributed buttons
`space-evenly`	Equal space everywhere including edges	Perfectly balanced layouts

> Run the interactive `flex-demo` to see all flex modes in action: `./run-demo.sh flex-demo`. Use arrow keys to cycle through modes and number keys (1-4) to switch between examples.

Programmatic Usage

Use flex with `Row` and `Column` elements in the Toolkit DSL:

```
// Horizontal flex (Row)
Element horizontalFlex = row(
    panel(() -> text("Left")).rounded().length(10),
    panel(() -> text("Center")).rounded().length(10),
    panel(() -> text("Right")).rounded().length(10)
).flex(Flex.SPACE_BETWEEN);

// Vertical flex (Column)
Element verticalFlex = column(
    panel(() -> text("Top")).rounded().length(3),
```

```
    panel(() -> text("Middle")).rounded().length(3),
    panel(() -> text("Bottom")).rounded().length(3)
).flex(Flex.CENTER);

// Nested layouts with different flex modes
Element nestedFlex = column(
    row(
        text("Item A").length(8),
        text("Item B").length(8)
    ).flex(Flex.START).length(3),

    row(
        text("Item C").length(8),
        text("Item D").length(8)
    ).flex(Flex.END).length(3)
);
```

CSS Usage

Set flex via the `flex` property:

```
/* Center toolbar items */
.toolbar-row {
    flex: center;
}

/* Space out menu items */
.menu-row {
    flex: space-between;
}

/* Pack footer content to the right */
#footer-row {
    flex: end;
}

/* Vertical centering in a column */
.sidebar-column {
    flex: center;
}
```

```
// Elements pick up flex from CSS classes
Element toolbarRow = row(
    text("New"),
    text("Open"),
    text("Save")
).addClass("toolbar-row");
```

Flex vs Fill: Understanding the Difference

This is a common point of confusion. They serve different purposes:

- `flex` : Controls *where* items are positioned in remaining space
  - Only affects positioning of items that don't fill the container
  - Applied to the container (Row/Column)
  - Example: `row(…).flex(Flex.CENTER)`
- `fill()` : Controls *how much* space an item should consume
  - Makes an item grow to take available space
  - Applied to individual children
  - Example: `text("grows").fill()`

```
// WITHOUT fill() - flex positions fixed-size items
Element withoutFill = row(
    text("A").length(5),
    text("B").length(5),
    text("C").length(5)
).flex(Flex.SPACE_BETWEEN);  // Items stay 5 wide, spread out

// WITH fill() - item grows, flex has less effect
Element withFill = row(
    text("A").length(5),
    text("B").fill(),          // Takes all remaining space
    text("C").length(5)
).flex(Flex.CENTER);            // Only affects tiny leftover gaps
```

Practical Examples

Centered Dialog

```
Element centeredDialog = column(
    panel("Confirm",
        column(
            text("Are you sure?"),
            row(
                text("[ Yes ]").green(),
                text("[ No ]").red()
            ).flex(Flex.SPACE_AROUND).spacing(2)
        )
    ).rounded().length(10)
).flex(Flex.CENTER);
```

Application Layout

```
Element appLayout = column(
    // Header: left-aligned title
    row(
        text("My App").bold(),
        text("v1.0").dim()
    ).flex(Flex.START).spacing(2).length(3),

    // Content: fills available space
    panel(() -> text("Main content...")).fill(),

    // Footer: spread items across
    row(
        text("Ready").green(),
        text("Line 42, Col 15").dim(),
        text("UTF-8").dim()
    ).flex(Flex.SPACE_BETWEEN).length(1)
);
```

Toolbar with CSS

```
.toolbar {
    flex: space-between;
    spacing: 1;
```

```
}

.toolbar-left {
    flex: start;
}

.toolbar-right {
    flex: end;
}
```

```
// Main toolbar with items spread across
Element mainToolbar = row(
    text("File"),
    text("Edit"),
    text("View"),
    text("Help")
).addClass("toolbar");

// Toolbar section with items grouped left
Element leftToolbar = row(
    text("New"),
    text("Open")
).addClass("toolbar-left");
```

Flex with Spacing

Flex and spacing work together. Spacing creates fixed gaps between items, then flex distributes any remaining space:

```
// spacing=1 creates 1-cell gaps, flex centers the whole group
Element spacedRow = row(
    text("A"),
    text("B"),
    text("C")
).spacing(1).flex(Flex.CENTER);
```

Common Patterns

```
// Left-aligned button group
Element leftAligned = row(
    text(" Save ").bold().black().onGreen(),
    text(" Cancel ").bold()
).flex(Flex.START).spacing(1);

// Right-aligned status
Element rightAligned = row(
    text("Ready").green()
).flex(Flex.END);

// Split layout: title left, controls right
Element splitLayout = row(
    text("Settings").bold().length(20),
    spacer(),  // Takes remaining space
    text(" OK ").bold().length(6),
    text(" Cancel ").bold().length(8)
).flex(Flex.START).spacing(1);

// Perfectly centered modal
Element centeredModal = column(
    spacer(),
    row(
        spacer(),
        panel("Notice",
            () -> text("Operation completed")
        ).rounded().length(30),
        spacer()
    ),
    spacer()
);
```

Layout Example

This example shows how to use CSS layout properties to create a centered footer:

```
/* Center the footer content */
.footer-row {
    flex: center;
}

/* Size text to content width, allowing flex to center */
.footer-row .title {
```

```
    width: fit;
}

.footer-row .dim {
    width: fit;
}

/* Fixed heights for header/footer panels */
.header-panel {
    height: 3;
}

.footer-panel {
    height: 3;
}

/* Main content fills remaining space */
.main-content {
    height: fill;
}
```

```
Element layout = column(
    panel(() -> header()).addClass("header-panel"),
    panel(() -> content()).addClass("main-content"),
    panel(() -> row(
        text("Status: ").addClass("title"),
        text("Ready").addClass("dim")
    ).addClass("footer-row")).addClass("footer-panel")
);
```

Named colors: `black`, `red`, `green`, `yellow`, `blue`, `magenta`, `cyan`, `white`, `gray`, `dark-gray`, and their bright variants.

Importance

Override specificity with `!important`:

```
.error {
    color: red !important;
```

```
}
```

The Property System

TamboUI uses a typed property system where each CSS property is defined with its type, converter, and inheritance behavior.

PropertyDefinition

A `PropertyDefinition<T>` bundles the property name with its converter and metadata:

```
// Simple non-inheritable property
PropertyDefinition<Color> GAUGE_COLOR =
    PropertyDefinition.of("gauge-color", ColorConverter.INSTANCE);

// Inheritable property with default value
PropertyDefinition<Color> COLOR =
    PropertyDefinition.builder("color", ColorConverter.INSTANCE)
        .inheritable()
        .build();
```

PropertyRegistry

The `PropertyRegistry` is a registry where all property definitions must be registered for the style system to recognize them:

```
// Register a single property
PropertyRegistry.register(MY_PROPERTY);

// Register multiple properties at once
PropertyRegistry.registerAll(PROPERTY_A, PROPERTY_B, PROPERTY_C);
```

When a widget defines custom properties, it should register them in a static initializer block:

```java
public static class MyWidget {
    public static final PropertyDefinition<Color> BAR_COLOR =
        PropertyDefinition.of("bar-color", ColorConverter.INSTANCE);
    public static final PropertyDefinition<Integer> BAR_WIDTH =
        PropertyDefinition.of("bar-width", intConverter());

    static {
        PropertyRegistry.registerAll(BAR_COLOR, BAR_WIDTH);
    }

    private static PropertyConverter<Integer> intConverter() {
        return value -> Optional.of(Integer.parseInt(value));
    }
}
```

Unregistered properties may trigger warnings or errors depending on the style engine configuration.

StandardProperties

The `StandardProperties` class defines all built-in core properties. It serves as the central registry for properties that all widgets can use:

Property	Type	Inherits	Description
`color`	Color	Yes	Foreground/text color
`text-style`	Set<Modifier>	Yes	Text modifiers (bold, dim, italic, etc.)
`border-type`	BorderType	Yes	Border style (plain, rounded, double, thick)
`background`	Color	No	Background color
`border-color`	Color	No	Border color

Property	Type	Inherits	Description
`padding`	Padding	No	Inner spacing
`margin`	Margin	No	Outer spacing
`width` , `height`	Constraint	No	Size constraints
`flex` , `direction` , `spacing`	Various	No	Layout properties

Property Inheritance

In TamboUI, elements form a tree structure: a `Panel` contains `Text` elements, which may contain `Span` elements, and so on. *Inheritance* determines whether a property value automatically flows from a parent element to its children.

Inherited Properties

When you set an inherited property on a parent, all descendants automatically receive that value unless they explicitly override it.

```
Panel.sidebar {
    color: cyan;        /* Inherited - flows to all children */
    text-style: dim;    /* Inherited - flows to all children */
}
```

```
// All text inside the sidebar automatically gets cyan + dim styling
Element sidebar = panel(() -> column(
    text("Menu"),           // cyan, dim (inherited from Panel)
    text("Dashboard"),      // cyan, dim (inherited from Panel)
    text("Settings").bold() // cyan, dim + bold (inherits color, adds bold)
)).addClass("sidebar");
```

The inherited properties are: `color`, `text-style`, `border-type`.

Non-Inherited Properties

Non-inherited properties only apply to the element they're set on. Children must set these properties explicitly.

```
Panel.sidebar {
    background: dark-gray;  /* NOT inherited - only Panel gets this */
    padding: 1;             /* NOT inherited - only Panel gets this */
}

/* Children need their own background if desired */
Panel.sidebar Text {
    background: black;      /* Must be set explicitly on Text */
}
```

This matches standard CSS behavior: setting a background on a container doesn't automatically give all its children that same background.

The non-inherited properties include: `background`, `padding`, `margin`, `width`, `height`, `flex`, `direction`, `spacing`.

Widget-Specific Properties

Widgets can define their own properties for custom styling. For example, the `Gauge` widget defines:

```
public static final PropertyDefinition<Color> GAUGE_COLOR =
    PropertyDefinition.of("gauge-color", ColorConverter.INSTANCE);
```

This allows CSS like:

```
GaugeElement {
    gauge-color: green;
}
```

See the Developer Guide for details on creating widgets with custom properties.

Applying Styles

With Toolkit DSL

Elements automatically participate in CSS styling when you set IDs and classes:

```
Element settingsPanel = panel("Settings",
    () -> column(
        text("Username").addClass("label"),
        textInput(usernameState).id("username-input"),
        text("Password").addClass("label"),
        textInput(passwordState).id("password-input").addClass("secret")
    )
)
.id("settings-panel")
.addClass("primary");
```

With ToolkitRunner

Pass the StyleEngine when creating the runner:

```
StyleEngine engine = StyleEngine.create();
engine.loadStylesheet("dark", "/themes/dark.tcss");
engine.setActiveStylesheet("dark");

try (var runner = ToolkitRunner.builder()
        .styleEngine(engine)
        .build()) {
    runner.run(() -> myApp());
}
```

Implementing Styleable

For custom widgets outside the Toolkit, implement the `Styleable` interface:

```java
public static class MyStylableWidget implements Styleable {
    private String id;
    private Set<String> classes = new HashSet<>();

    @Override
    public String styleType() {
        return "MyWidget";  // Used for type selectors
    }

    @Override
    public Optional<String> cssId() {
        return Optional.ofNullable(id);
    }

    @Override
    public Set<String> cssClasses() {
        return classes;
    }

    @Override
    public Optional<Styleable> cssParent() {
        return Optional.empty();  // Or return parent for descendant selectors
    }
}
```

Then resolve and apply styles:

```java
CssStyleResolver resolved = engine.resolve(widget);

Style style = Style.EMPTY;
if (resolved.foreground().isPresent()) {
    style = style.fg(resolved.foreground().get());
}
if (resolved.background().isPresent()) {
    style = style.bg(resolved.background().get());
}
for (var modifier : resolved.modifiers()) {
    style = style.addModifier(modifier);
}
```

Theme Switching

Switch themes at runtime:

```
void onToggleTheme(StyleEngine engine) {
    String current = engine.getActiveStylesheet().orElse("dark");
    String next = "dark".equals(current) ? "light" : "dark";
    engine.setActiveStylesheet(next);
}
```

Listen for style changes:

```
engine.addChangeListener(() -> {
    // Styles changed, trigger redraw
    requestRedraw();
});
```

Example Theme Files

dark.tcss

```
$bg-primary: black;
$fg-primary: white;
$accent: cyan;
$border-color: dark-gray;

* {
    color: $fg-primary;
    background: $bg-primary;
}

Panel {
    border-type: rounded;
    border-color: $border-color;
}
```

```
Panel:focus {
    border-color: $accent;
    border-type: double;
}

.primary {
    color: $accent;
    text-style: bold;
}

.danger {
    color: red;
}

.muted {
    color: gray;
}
```

light.tcss

```
$bg-primary: white;
$fg-primary: black;
$accent: blue;
$border-color: gray;

* {
    color: $fg-primary;
    background: $bg-primary;
}

Panel {
    border-type: rounded;
    border-color: $border-color;
}

Panel:focus {
    border-color: $accent;
    border-type: double;
}

.primary {
    color: $accent;
```

```
    text-style: bold;
}
```

Next Steps

- API Levels - understanding the different abstraction layers
- Widgets Reference - available components to style
- Developer Guide - creating custom styleable widgets

PREVIOUS

API Levels

NEXT

Bindings and Actions